

Урок 18. Выбор темы, структура. Разработка API с помощью Декораторы запросов – подробнее (то что у нас декоратор @app.get())

1) Внешний вид карточки метода

- `summary`: короткий заголовок операции → **виден** и в `/docs`, и в `/redoc`.
- `description`: подробное описание (поддерживает Markdown) → **видно** в обоих.
- `tags`: `list[str]` → группировка эндпоинтов по разделам → **видно** в обоих.
- `deprecated`: `bool` → пометка «deprecated» → **видно** в обоих.
- `include_in_schema`: `bool` → если `False`, эндпоинт вообще **не попадает** ни в `/docs`, ни в `/redoc`.
- `operation_id`: `str | None` → технический ID операции (иногда показывается в Swagger как «OperationId», ReDoc чаще скрывает) → **обычно не бросается в глаза**.
- `name`: `str | None` → внутреннее имя маршрута (для `url_path_for`), **в доках не показывается**.

2) Ответы и их схемы

- `response_model`: модель/тип ответа → на основе этого строится **схема ответа** и примеры в `/docs` и `/redoc`.
- `status_code`: `int` → код «по умолчанию» (например, 200) → **виден** как основной блок ответа.
- `response_description`: `str` → подпись к основному ответу → **видна**.
- `responses`: `dict[int | str, ...]` → доп. ответы (404/422 и т.п.) с описаниями/примерами → **видны** в обоих.
- Флаги сериализации (`response_model_exclude_unset`, `response_model_exclude_defaults`, `response_model_exclude_none`, `response_model_include/exclude`, `response_model_by_alias`) → **на рендер напрямую не выводятся**, но влияют на то, что реально вернётся клиенту (и на пример в Swagger, если он собирается из реального ответа).

В названии декоратора сначала идет название объекта приложения, которое мы создаем строчкой `app = FastAPI()`, а затем указывается запрос, который мы хотим создать.

Запросы, которые вносят изменения

Основные запросы

- **POST** — создаёт *новый ресурс* в коллекции. Обычно **сервер сам выдаёт id**. Неидемпотентен (два одинаковых POST → два объекта).
- **PUT** — *полностью создаёт/заменяет ресурс по известному URI*. Идемпотентен (повтор одного и того же PUT оставит состояние тем же).

- **PATCH** — *частичное обновление* ресурса (меняем только пару полей, а не всю запись).
- **DELETE** — удаляет ресурс по URI. Идемпотентен (повтор не меняет состояние).

Реализация в проекте

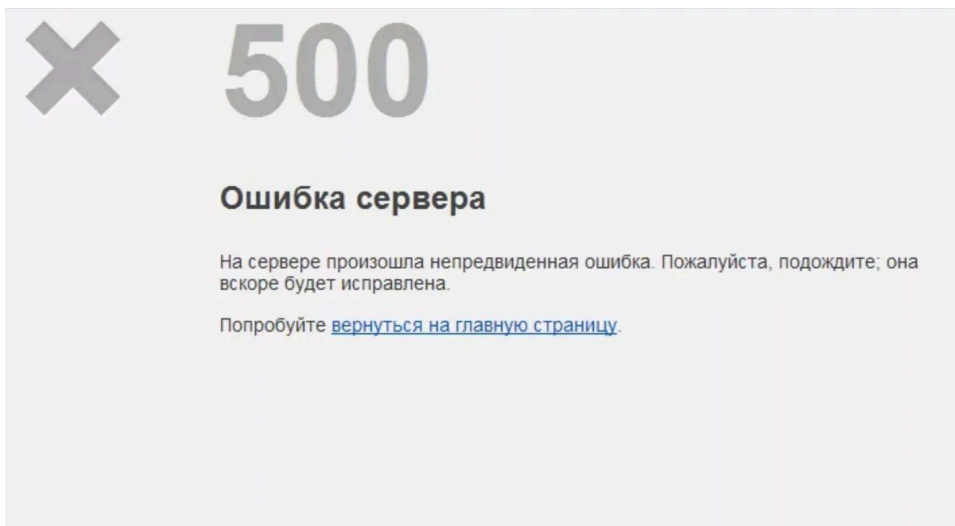
- **Создать ученика** → **POST /students**
Тело без `student_id` (сервер сгенерирует). Обычный ответ: 201 Created + созданный объект, если объект уже существует — 409, стандартно если весь объект `students.json` не существует — 500.
- **Полностью заменить карточку** (все поля) → **PUT /students/{student_id}**
Тело — полная модель `Student`. Повторы безопасны (идемпотентность). Ошибки: несовпадение по `id` — 400, если объект не найден — 404, стандартно если весь объект `students.json` не существует — 500.
- **Изменить только класс (grade)** → лучше **PATCH /students/{student_id}** с телом `{"grade": 5}`. Если **PATCH** не хочется — допустимо смоделировать подресурс: **PUT /students/{id}/grade** с телом `{"grade": 5}`. Это ок, но менее «канонично», чем **PATCH**. Ошибки: если объект не найден — 404, стандартно если весь объект `students.json` не существует — 500.
- **Удаление ученика** → **DELETE /students/{student_id}**. Обычный ответ: **204 No Content**; если ресурса нет — **404 Not Found** (иногда тоже 204)

Коды запроса и ошибки

Зачем прописывать ошибки и проверять их? Ошибки вносятся в параметр запроса в `responses`, что войдет в документацию как разъяснение, какой ответ мы ожидаем при разных ситуациях, также проверки и возвращение кода ошибки прописываются непосредственно в методе, на примере прошлых уроков:

```
try:
    students = json_to_dict_list(DATA)
except FileNotFoundError:
    raise HTTPException(status_code=500, detail="students.json not found")
```

raise в данном случае работает так, что с помощью него можно возвращать исключение, детализируя его по форме для API. Вспомните, наверняка в вашей жизни были ситуации, когда веб страница вам выкидывала ошибку 500.



Чаще 500 – это просто какая-либо внутренняя ошибка, широко применяема, например если какой-то функционал заявлен, но не реализован. В нашем случае эта ошибка просто сигнализирует об отсутствии json файла. Остальные коды состояний запроса (положительные: 200, 201, 204, отрицательные: 400, 404, 409) более детализированы и узконаправлены.

Пишем POST/PUT/PATCH/DELETE запросы по шаблону:

```
@app.post(
    "/students",
    tags=["students"],
    summary="Создать ученика (POST)",
    description="Принимает полную модель Student. Если student_id уже существует — 409 Conflict.",
    status_code=201,
    response_model=Student,
    responses={
        201: {"description": "Создано"},
        409: {"model": Error, "description": "Ученик с таким ID уже есть"},
        500: {"model": Error, "description": "Файл students.json не найден"},
    },
)
def create_student(payload: Student): # payload — это Pydantic-модель Student (валидируется Pydantic)

    -code-
    raise HTTPException(status_code=500, detail="students.json not found")

    -code-
    raise HTTPException(status_code=409, detail="student_id already exists")

    # в Pydantic v2 у моделей нет .dict(), вместо этого — .model_dump().
    # метод возвращает обычный словарь Python (готовый к сериализации в JSON).
    # мы добавляем сформированный словарь в список students, т.е. подготавливаем новые данные «в памяти».

    students.append(payload.model_dump())
    DATA.parent.mkdir(parents=True, exist_ok=True)
    dict_list_to_json(students, DATA)
    return payload

@app.put(
```

```

    "/students/{student_id}",
    tags=["students"],
    summary="Полная замена карточки (PUT)",
    description="Заменяет запись целиком. ID в пути и в теле должны
совпадать.",
    response_model=Student,
    responses={
        400: {"model": Error, "description": "Несовпадение ID"},
        404: {"model": Error, "description": "Ученик не найден"},
        500: {"model": Error, "description": "Файл students.json не найден"},
    },
)

def replace_student(student_id: int, payload: Student):
    -code-
        raise HTTPException(status_code=500, detail="students.json not found")

    -code-
        raise HTTPException(status_code=400, detail="student_id in path and
body must match")

    -code-
        return -code-
    raise HTTPException(status_code=404, detail="student not found")

@app.patch(
    "/students/{student_id}",
    tags=["students"],
    summary="Частичное обновление (PATCH)",
    description="Обновляет только переданные поля. Остальные остаются как
были.",
    response_model=Student,
    responses={
        404: {"model": Error, "description": "Ученик не найден"},
        500: {"model": Error, "description": "Файл students.json не найден"},
    },
)

def patch_student(student_id: int, patch: StudentUpdate): #см класс
StudentUpdate ниже
    -code-
        raise HTTPException(status_code=500, detail="students.json not found")

    -code-
        return -code-

    raise HTTPException(status_code=404, detail="student not found")

@app.delete(
    "/students/{student_id}",
    tags=["students"],
    summary="Удалить ученика (DELETE)",
    description="Удаляет запись по ID. Возвращает 204 No Content при успехе.",
    status_code=204,
    responses={
        204: {"description": "Удалено"},
        404: {"model": Error, "description": "Ученик не найден"},
        500: {"model": Error, "description": "Файл students.json не найден"},
    },
)

def delete_student(student_id: int):
    -code-
        raise HTTPException(status_code=500, detail="students.json not found")

```

```
-code-
        return Response(status_code=204) # подключите ответы в строке from
fastapi import

    raise HTTPException(status_code=404, detail="student not found")
```

Для удобства обновления полей нашего json, добавим новую модель в файл schema.py:

```
class StudentUpdate(BaseModel):
    """Тело для PATCH: все поля опциональны; валидируются, если переданы."""
    first_name: str | None = None
    last_name: str | None = None
    date_of_birth: str | None = None
    email: str | None = None
    phone_number: str | None = None
    address: str | None = None
    enrollment_year: int | None = None
    grade: int | None = Field(None, ge=1, le=11, description="Новый класс (1-11)")
    special_notes: str | None = None
```